DTU Compute
Department of Applied Mathematics and Computer Science

Occupancy-Aware Climate Control Estimating Thermal Comfort Curves with Hidden Markov Models

Bachelor Thesis



Occupancy-Aware Climate Control
Estimating Thermal Comfort Curves with Hidden Markov Models

Bachelor Thesis
7 December, 2026

By
Mads Holme Håkonsson

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.
Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Asmussens Alle, Building 322, 2800 Kgs. Lyngby Denmark
https://www.compute.dtu.dk
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Approval

This thesis has been prepared over six months at the Section for Indoor Climate, Department of Civil Engineering, at the Technical University of Denmark, DTU, in partial fulfilment for the degree Master of Science in Engineering, MSc Eng.

It is assumed that the reader has a basic knowledge in the areas of statistics.

Mads Holme Håkonsson - s234936

.....

Signature

.....

Date

Abstract

ABSTRACT IS MISSING

Acknowledgements

Mads Holme Håkonsson, MSc Civil Engineering, DTU
Creator of this thesis template.

[Name], [Title], [affiliation]
[text]

[Name], [Title], [affiliation]
[text]

Contents

Preface . . . ii
Abstract . . . iii
Acknowledgements . . . iv

1 Introduction **1**

2 Data **2**

3 Theory **6**
3.1 Modelling CO_2 dynamics . . . 6
3.1.1 The underlying physical model & assumptions . . . 6
3.1.2 The ODE as a diffusion Process (Ornstein-Uhlenbeck) . . . 7
3.1.3 Properties of the Ornstein-Uhlenbeck process . . . 7
3.2 Hidden Markov Model (HMM) theory . . . 8
3.2.1 HMM Models . . . 8
Generel Theory . . . 8
HMM(1)-Model . . . 8
HMM($\beta, 1$)-model . . . 9
AR(1)-HMM(1) . . . 9
AR(1)-HMM($\beta, 1$) . . . 10
3.2.2 Likelihood . . . 10
3.2.3 Backward Sampling . . . 11
3.2.4 Model Diagnostics . . . 11
Goodness-of-fit . . . 11
Test Statistics for Comparison . . . 12

4 Implementation **13**
4.1 Coding envoriment . . . 13
4.2 Class Design . . . 14

5 Model Results **16**

6 Simulation Study **17**

7 Discussion **18**

8 Conclusion **19**

9 Future Work **20**

Bibliography **21**

A Proof of the AR(1) representation **23**

1 Introduction

Reduction in the usage of energy in buildings is a crucial aspect if one wants to go towards a more sustainable future. It has long been known that buildings are responseible for X% of the total energy consumption in the world, and that a large part of this energy is for HVAC systems [MISSING REFERENCE] HVAC systems energy consumption has been directly related to the occupant behaviours of the buildings. This imposes a challenge, as one has to be able to understand the behaviour of the occupants in the buildings before we can gain the largest optimization in energy consumption. (MISSING REFERENCE) has shown that there is a large variability in the behavour of occupants such as thermal comfort, lighting & ventiliation needs and precense hours.

Within the context of energy consumption in buildings, heating has been estimated to contribute to X% [MISSING REFERENCE] of the consumption. An aspect of heating of a building is the byproduct of the thermal comfort of the occupants in the building, and therefore, if we can understand the thermal comforts of the occupants in the buildings, we can optimize the energy consumption drasitically. However, the thermal comfort curves is a personal preference of the occupants, and we should therefore expect this to differer dependent on the individual buildings and the number of occupants.

This projects aims to understand when the occupants of a single offices are present, and how much thermal comfort feedback does we need to collect from the occupants in order to be able to infer the true thermal comfort of the occupants in the building. The goal is to also build a codebase others and easyly download and extent.
We will do this by using a Hidden Markov Model (HMM) to model the state of the room

WRITTEN OUTLINE OF EACH CHAPTER AND WHAT QUESTIONS THEY ANSWER

2 Data

The data used in this project are CO_2 data collected from the office building 303B and the censors and data providers are climify [MISSING REFERENCE]. The data is collected in the fall and winter period of 2023, and the raw data period spans from [MISSING REFERENCE] to [MISSING REFERENCE]. The data collected from the censors are in intervals of around 10 minutes. Because our HMM models works in discrete time, the data have been aggregated into 30 minutes such that each time step $\Delta t = 1$ represents 30 minutes.

When collecting CO_2 data, the censors tend to drift over time [MISSING REFERENCE], and therefore, the data collected from the censors are not always accurate. To empty rooms, that have the same CO_2 levels may show different readins based on how much the censors have drifted over time. This is a problem, as we want to use the CO_2 data to infer the presence of the occupants in the building and a higher baseline of the CO_2 data may lead to a false positive of the presence of the occupants in the building. To correct for this, we have used a method called *Automatic Baseline Correction* (ABC) Algorithm. [MISSING REFERENCE]. In the use of this algorithm, we have assumed that the lowest CO_2 ppm values over a period is equal to the outside CO_2 ppm values. From literature, it's common to set the outside CO_2 ppm values to 400 ppm [MISSING REFERENCE], which we have also done.

Let C_t be the CO_2 ppm value at time t, $\tilde{C}_t$ be the corrected CO_2 ppm value at time t.

We then define the modified ABC algorithm as

$$\tilde{C}_t = C_t - Q_{0.02}\left(C_{t-\Delta t:t}\right) + 400, \tag{2.1}$$

where $Q_{0.02}\left(C_{t-\Delta t:t}\right)$ is the 2% quantile of the CO_2 ppm values over the time period $[t - \Delta t, t]$ and Δt is a fixed time period. In this project, we have set $\Delta t = 8$ days, such that the weekday overlaps. The reason for using a 2% quantile is to avoid the effect of erradic readings of the CO_2 ppm values, which may occur if the censors are not working properly. Because we are interrested in predicting future presence of the occupants in the building,

we dont need the data to be as granular as 10 minutes.

Room	N	Min	2.5%	Median	Mean	97.5%	Max
Room 001	14289	341.00	426.00	621.00	828.52	2411.00	3185.00
Room 003	14328	335.00	405.00	516.00	611.92	1424.00	3025.00
Room 004	14268	292.00	400.00	940.50	1201.19	3042.00	3710.00
Room 005	14324	345.00	418.00	704.00	774.84	1500.00	2447.00
Room 007	14334	360.00	414.00	633.00	736.45	1588.00	2805.00
Room 008	14321	360.00	429.00	781.00	988.87	2824.00	3581.00
Room 009	14346	325.00	417.00	685.00	935.13	2322.50	3654.00
Room 012	14344	335.00	414.00	739.00	769.75	1406.00	2860.00
Room 013	14363	247.00	442.00	716.00	852.76	1841.00	2579.00
Room 014	14144	327.00	422.00	804.00	1098.36	3305.42	4000.00
Room 015	14336	337.00	414.00	754.50	985.40	2706.25	3808.00
Room 016	14333	349.00	422.30	686.00	833.00	1841.70	3825.00
Room 019	14338	355.00	435.00	727.00	886.71	1927.57	2756.00

Table 2.1: Summary statistics of the raw CO_2 readings (ppm) per room, after removing missing, saturated and duplicate readings.

TO DO:

- Make summary table of the data with before and after the ABC algorithm (n, mean, sd, min, quantiles, max, % of bins below 450 ppm; one row per room if several rooms).
- Make ACF plots to show the AR structure of the data. Add PACF (shows AR order); note daily cycle at lag 48.
- Make a plot of the data before and after the ABC algorithm (raw, estimated floor, calibrated + zoom on one week).
- Histogram/density of calibrated CO_2 (multimodality motivates hidden states).
- Average daily profile, weekday vs. off-day (motivates off-day + Fourier covariates).
- Overview plot of full series with train/val/test boundaries and feedback events.
- Short chapter introduction and conclusion.

Fix in existing text:

- Δt used both as 30-min step and as 8-day window: rename window to W.
- Dates: Room 009 segment is 2023-11-21 to 2024-02-09 (3860 bins, $\approx$ 80 days), not "fall and winter 2023". Clarify 303B vs. DTU dataset (live `room_009/30min` data comes from `clean_dtu_data.py`).
- Reason for 8 days: every window contains a full weekend (an empty-room period).
- Merge the duplicated aggregation paragraph; state that bins are means of the readings.
- Typos: censors $\rightarrow$ sensors, "To empty rooms" $\rightarrow$ Two, readins, erradic, interrested, dont, climify $\rightarrow$ Climify.
- Define a `\COtwo` macro instead of retyping `$\text{CO}_2$`.

Missing processing steps to describe:

- Raw cleaning: drop NaNs, keep 0–4000 ppm (sensor saturation), remove duplicates.
- Gap handling: only the longest gap-free segment is kept (discrete model needs equal spacing).
- ABC details: applied after 30-min binning (and why); trailing window, so no leakage into val/test; first 8 days dropped as warm-up; justify 2% quantile (must be below share of empty bins, $\approx$16% below 500 ppm in Room 009).
- Report floor drift (Room 009: 388 $\rightarrow$ 500 ppm).
- State ABC assumption: room is empty and ventilated to outdoor level at some point in every window. Outdoor CO_2 was $\approx$420 ppm in 2023–24: justify 400 or mention it.
- Feedback data: how collected, number of responses, binary flag per 30-min bin.
- Humidity: calibration against outdoor humidity, or state that it is excluded.

- Covariates: off-day flag (weekends + holidays), daily (3 harmonics) and weekly (1 harmonic) Fourier terms, weather variables; z-scored with training statistics only.
- Time zone: timestamps in UTC (no DST change in Nov–Feb).
- Split: chronological 65/15/20, boundaries 2024-01-12 and 2024-01-24; Christmas break falls in training.
- Rooms and sensors: which rooms and why, single/shared offices, ventilation type, sensor type (NDIR), accuracy, sampling interval.

Missing references: Climify, NDIR sensor drift, ABC (Senseair firmware; check whether your sensors already run it), outdoor CO_2 baseline (e.g. NOAA).

3 Theory

In this chapter, a ODE of the physical model of the CO_2 dynamics will be presented and how the process is descibed as a stochastic differential equation (SDE), where noise has been added to the system.
Given the presented SDE, it will be shown that the process can be desbribed as a discrete-time AR(1) process, where the production rate of the occupants in the room is modelled as a function of the occupancy state of the room.
Hereafter, the generel theory of the hidden markov models (HMM) will be presented along with the inference algorithms used in this project.

3.1 Modelling CO_2 dynamics

This subsection present the underlying physical model of the CO_2 dynamics of a room and show that it can be described as a discrete-time AR(1) process and how the production rate of the occupants in the room can be modelled as a function of the occupancy state of the room.

3.1.1 The underlying physical model & assumptions

Due to CO_2 does not decay by itself, the perfect mixtured mass balance model of a room (Batterman 2017; Wang, Burnett, and Chong 1999) is given as a first-order ordinary differential equation (ODE) as

$$V\frac{dC_{\text{indoor}}(t)}{dt} = G(t) + Q\left(C_{\text{outdoor}}(t) - C_{\text{indoor}}(t)\right), \tag{3.1}$$

where V is the room volume (m^3), C_{indoor} and C_{outdoor} are the indoor and outdoor CO_2 concentrations ($mg{\cdot}m^{-3}$), $G(t)$ is the occupant production rate ($mg{\cdot}h^{-1}$) and Q is the outdoor air flow rate ($m^3{\cdot}h^{-1}$).

Dividing by V gives

$$\frac{dC_{\text{indoor}}(t)}{dt} = \frac{G(t)}{V} - \frac{Q}{V}\left(C_{\text{indoor}}(t) - C_{\text{outdoor}}(t)\right). \tag{3.2}$$

Assuming C_{outdoor} is constant, the excess concentration $\tilde{C}(t) = C_{\text{indoor}}(t) - C_{\text{outdoor}}$ satisfies $d\tilde{C}/dt = dC_{\text{indoor}}/dt$, so

$$\frac{d\tilde{C}(t)}{dt} = \frac{G(t)}{V} - \frac{Q}{V}\tilde{C}(t). \tag{3.3}$$

The sensors report ppm, related by $C\,[\text{ppm}] = C\,[\text{mg}\cdot\text{m}^{-3}]\cdot V_m/M_{CO_2}$, with $M_{CO_2} = 44.01$ $g{\cdot}mol^{-1}$ and $V_m \approx 24.45$ $L{\cdot}mol^{-1}$ (25°C, 1 atm). Multiplying by the constant V_m/M_{CO_2} gives the model in ppm,

$$\frac{d\tilde{C}(t)}{dt} = P(t) - \alpha\tilde{C}(t), \qquad P(t) = \frac{G(t)\,V_m}{M_{CO_2}\,V}, \qquad \alpha = \frac{Q}{V}, \tag{3.4}$$

where $\tilde{C}(t)$ is now the excess CO_2 concentration (ppm), $P(t)$ is the production rate ($ppm{\cdot}h^{-1}$) and α is the air change rate (h^{-1}), i.e. the fraction of excess CO_2 ventilated out per hour.

The assumptions in 3.1, and therefore in 3.4, are that

- The CO_2 in the room is perfectly mixed, meaning that the CO_2 is evenly distributed in the room. This ensures that the sensors are not biased by location.
- There is no CO_2 sinks in the room, meaning that the CO_2 is not absorbed by any materials in the room and is only ventilated out of the room.

However, it's important to note that the model in 3.4 is a simplification of the real-world CO_2 dynamics of a room, and that there are many factors that can affect the CO_2 levels in a room, such as the number of occupants, the activity level of the occupants, the ventilation system, and the outdoor CO_2 levels.
To be able to model the production rate of the occupants in the room, we will assume that the production rate $P(t)$ is a stepwise function. Let $S(t) = k \in \mathbb{N}$ be the number of occupants of the room at time t, then we can model the production rate as

$$P(t) = \beta_{S(t)}, \tag{3.5}$$

where $\beta_{S(t)}$ is the production rate of the occupants in the room,

One of the main limitations of the model is that it does not assume noise in the system, which is invenatiable as we use sensors.
Therefore, the next step to make the model more realistics is to add noise to the system in form of a stochastic differential equation (SDE), which will be presented in the next subsection.

3.1.2 The ODE as a diffusion Process (Ornstein-Uhlenbeck)

As previous described, the model in 3.4 desvibe the dynamics of the CO_2 in a room, but it does not take into account the noise in the system.
By adding noise to the system in form of a linear diffusion tern with constant diffusion coefficient σ, we can write the model as a stochastic differential equation (SDE) as

$$d\tilde{C(t)} = (P(t) - \alpha\tilde{C(t)})dt + \sigma dW(t), \tag{3.6}$$

where $W(t)$ is a standard Brownian motion.
3.6 is regornized as an Ornstein-Uhlenbeck process, which is a mean-reverting process, where the process tends to oscillate around some equilibrium state (chap. 1.13.1, Iacus 2008)

It is important to stress that the noise term in 3.6 was assumed to be a linear diffusion term with constant diffusion coefficient σ, as this is the simpelst noise to add to the system. However, one could also add a non-linear difussion term, where the diffusion coefficient is a function of the state of the system. This is however out of the scope of this project, as this would require more in depth knowledge of SDEs and stochastic calculus, which is not the focus of this project.

Next we show that, sampled at a fixed time step, the SDE in 3.6 is a discrete-time AR(1) process.

3.1.3 Properties of the Ornstein-Uhlenbeck process

Since 3.6 is an Ornstein-Uhlenbeck process, its solution is known in closed form (Iacus 2008, Sec. 1.13.1). Sampling it with a constant time step Δt and holding P constant over the step gives the recursion

$$\tilde{C}_{t+\Delta t} = e^{-\alpha \Delta t}\,\tilde{C}_t + \mu\left(1 - e^{-\alpha \Delta t}\right) + e_{t+\Delta t}, \qquad \mu = \frac{P}{\alpha}, \tag{3.7}$$

where μ is the equilibrium level the process reverts to, and the noise terms

$$e_{t+\Delta t} = \sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)}\, \mathrm{d}W_u \sim \mathcal{N}\left(0,\ \frac{\sigma^2}{2\alpha}\left(1 - e^{-2\alpha\Delta t}\right)\right) \tag{3.8}$$

have mean zero, constant variance and are uncorrelated over time. 3.7 is therefore an AR(1) process with autoregressive coefficient $\phi = e^{-\alpha\Delta t}$. The full derivation is given in Appendix A.
As $\alpha > 0$, from (chapter 1.13.1 Iacus 2008) it is known, that the process is ergodic the distribution of $\tilde{C}_t$ is

$$\tilde{C}_t \sim \mathcal{N}\left(\frac{P}{\alpha},\ \frac{\sigma^2}{2\alpha}\right), \tag{3.9}$$

which is the stationary distribution of the process.

3.2 Hidden Markov Model (HMM) theory

This section will present the underlying models and the theory behind it that models the diffusion process. Here to, inference algorithms and analysis methods will be presented.

3.2.1 HMM Models

Generel Theory
Hidden markov models are markov chains where the underlying states are hidden and only a random variable that is dependent on the underlying state is observed.
This can be described as a markov chain with a set of hidden states $S = \{s_1, s_2, ..., s_N\}$ and a set of observations $O = \{o_1, o_2, ..., o_M\}$. The transition between the hidden states is governed by a transition matrix Γ and the observations are generated by an emission matrix B. The initial state distribution is given by π. The HMM stucture can then debsrib the emissions as

$$P(O_t|S_t) = B_{S_t,O_t}, \tag{3.10}$$

where B_{S_t,O_t} is the probability of observing O_t given the hidden state S_t. The transition between the hidden states is given by

$$P(S_t|S_{t-1}) = \Gamma_{S_{t-1},S_t}, \tag{3.11}$$

where Γ_{S_{t-1},S_t} is the probability of transitioning from hidden state S_{t-1} to hidden state S_t. The initial state distribution is given by

$$P(S_1) = \pi_{S_1}, \tag{3.12}$$

where π_{S_1} is the probability of starting in hidden state S_1.

HMM(1)-Model
The ordinary HMM is the simplest model implemented in this project and the one most commonly used in the literature. The transition probabilities are constant and do not depend on time, the observations or any covariates, and neither does the emission density. For this model, we want to estimate the parameters $\theta = \{\Gamma, \mu, \sigma\}$, where $\Gamma \in \mathbb{R}^{n\times n}$ is the transition matrix, $\mu \in \mathbb{R}^n$ is the mean vector of the Gaussian emission density and $\sigma \in \mathbb{R}^n$ is the standard deviation vector of the Gaussian emission density. Because the Markov chain is finite, discrete and homogeneous, a unique strictly positive stationary distribution δ exists (chapter 1.3.2, Zucchini, MacDonald, and Langrock 2016) and is given by

$$\delta = \mathbf{1}^T \left(I - \Gamma + E\right)^{-1}, \tag{3.13}$$

where E is an $n \times n$ matrix of ones.

To handle the natural bounds on the standard deviations and the transition probabilities, the parameters are reparameterised. The standard deviations are given by

$$\sigma_i = \exp(\eta_i). \tag{3.14}$$

For the transition matrix, the $n(n-1)$ off-diagonal entries are parameterised. The diagonal is used as the reference category with log-odds fixed to zero, such that the logits form a matrix of the form

$$\xi = \begin{pmatrix} - & \xi_{12} & \cdots & \xi_{1n} \\ \xi_{21} & - & \cdots & \xi_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \xi_{n1} & \xi_{n2} & \cdots & - \end{pmatrix}. \tag{3.15}$$

Concretely, the unconstrained parameters are stored as a matrix $\xi \in \mathbb{R}^{n\times(n-1)}$ that holds the off-diagonal logits. The transition matrix is then obtained by exponentiating these logits in the off-diagonal positions of an identity matrix (so the diagonal is $\exp(0) = 1$) and row-normalising:

$$\Gamma_{ij} = \begin{cases} \dfrac{\exp(\xi_{ij})}{1 + \sum_{k\neq i} \exp(\xi_{ik})}, & j \neq i, \\ \dfrac{1}{1 + \sum_{k\neq i} \exp(\xi_{ik})}, & j = i. \end{cases} \tag{3.16}$$

The μ parameters are not bounded, but to avoid label switching between the Gaussian emission distributions, the means are ordered as $\mu_1 < \mu_2 < \cdots < \mu_n$ by the reparameterisation

$$\mu_1 = \zeta_1, \qquad \mu_i = \zeta_1 + \sum_{k=2}^{i} \exp(\zeta_k), \quad i = 2, \ldots, n, \tag{3.17}$$

where $\zeta \in \mathbb{R}^n$. The unconstrained parameters $\eta \in \mathbb{R}^n$, $\xi \in \mathbb{R}^{n\times(n-1)}$ and $\zeta \in \mathbb{R}^n$ are thus the quantities that are optimised during the fitting process.

HMM(β, 1)-model
In this model, we let the transition probabilities depend on covariates which can include time-of-day, weekday, type of day and etc. The motivation for this is the assumption, that the occupancy state of the room is dependent on the time-of-day and weekday, as the room is used for work.

Let $x_t \in \mathbb{R}^p$ be a vector of p covariates at time t. The transition logits is then defined as

$$\xi_{ij}(t) = \beta_{ij}^T x_t + \tilde{\xi}_{ij}, \qquad i \neq j, \tag{3.18}$$

where $\beta_{ij} \in \mathbb{R}^p$ is a vector of regression coefficients for the transition from state i to state j and $\tilde{\xi}_{ij}$ is the intercept term. The transition matrix is then obtained as in 3.16 and we now need denote the transition matrix as $\Gamma_{i,j}^t = P(S_t = j | S_{t-1} = i)$.

AR(1)-HMM(1)
The autoregressive HMM (AR(1)-HMM(1)) extends the ordinary HMM by letting the emission distribution depend on the previous observation. Conditional on being in state i at time t, the observation is modelled as

$$Y_t \mid X_t = i,\, Y_{t-1} = y_{t-1} \sim \mathcal{N}\left(\mu_i + \phi_i \left(y_{t-1} - \mu_i\right),\, \sigma_i^2\right). \tag{3.19}$$

Each state therefore has its own AR(1) coefficient $\phi_i \in (-1, 1)$ in addition to a mean and a standard deviation. This may be enough to capture the temporal correlation of the CO_2 signal that the ordinary HMM leaves in the pseudo-residuals. The transition matrix Γ, the means μ and the standard deviations σ are parameterised exactly as in the HMM(1)-model, and the AR coefficients are reparameterised as

$$\phi_i = 2\, s(\psi_i) - 1, \qquad s(z) = \frac{1}{1 + e^{-z}}, \tag{3.20}$$

where $s(\cdot)$ is the logistic sigmoid, to enforce the stationarity constraint $|\phi_i| < 1$. The unconstrained parameter $\psi_i \in \mathbb{R}$ is the one passed to the optimiser, with the inverse mapping

$$\psi_i = \text{logit}\left(\frac{\phi_i + 1}{2}\right). \tag{3.21}$$

AR(1)-HMM(β, 1)
This model is the combination of the AR(1)-HMM(1) and the HMM(β, 1)-model, where the transition probabilities depend on covariates as in and the emission distribution is autoregressive.

3.2.2 Likelihood

Several methods can be used to estimate the parameters of a HMM, where at some of the more common methods are the Baum-Welch algorithm [MISSING REFERENCE].
In this project, we will however use the method of maximum likelihood estimation (MLE) to estimate the parameters of the HMM with the forward-filtering algorithm [MISSING REFERENCE].

The forward algorithm implemented is called recursive filtering (Møller et al. 2024), as it constructs the current state distribution $p(X_t \mid Y_{1:t})$ recursively from the previous state distribution $p(X_t \mid Y_{1:t-1})$.

Algorithm 1 Recursive Filtering Forward Algorithm

Require: Observations $\mathbf{y} = (y_1, \ldots, y_T)$, covariates $\mathbf{x} = (x_1, \ldots, x_T)$, initial state distribution $u_0 \in \mathbb{R}^{1 \times N}$, HMM parameters θ
Ensure: Predicted distributions $\{u_t\}$, filtered distributions $\{u_{t|t}\}$, step likelihoods $\{f_t\}$

for $t = 1, \ldots, T$ **do**
  $\Gamma_t \leftarrow \text{TransitionMatrix}(\theta, t, y_t, x_t)$ ▷ $N \times N$ transition matrix
  $u_{t|t-1} \leftarrow u_{t-1|t-1} \cdot \Gamma_t$ ▷ Predicted state distribution, $1 \times N$
  $g_t \leftarrow \text{Density}(\theta, t, y_t, x_t)$ ▷ Emission densities for all states, $1 \times N$
  $f_t \leftarrow \sum_{i=1}^{N} [u_t \odot g_t]_i$ ▷ Marginal likelihood at time t
  $f_t \leftarrow \max(f_t, 10^{-10})$ ▷ Clip to avoid division by zero
  $u_{t|t} \leftarrow \dfrac{u_t \odot g_t}{f_t}$ ▷ Filtered state distribution (normalized)
end for
return $\{u_t\}$, $\{u_{t|t}\}$, $\{f_t\}$

where $\odot$ denotes element-wise multiplication of vectors. The log-likelihood is obtained by summing the log of the step likelihoods:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \log f_t. \tag{3.22}$$

Each likelihood step f_t is the sum of the predicted state distribution u_t multiplied by the emission densities g_t for all states, which gives the marginal likelihood of observing y_t given the previous observations $y_{1:t-1}$.

3.2.3 Backward Sampling

When sampling a path from a hidden markov model, we can use the method defined in (chapter. 7.2.1, Zucchini, MacDonald, and Langrock 2016) and we will call it backward sampling.

We want to sample a path of the hidden states $S_{1:T}$ given the observations $O_{1:T}$ and the model parameters θ. This method is very useful, as it does not depend on the inital state distribution π, which is often unknown.

Let $u_{t|t} = P(S_t|O_{1:t}, \theta)$ be the filtered state distribution. We can then sample the last state S_T from the filtered distribution $u_{T|T}$. We can then sample the previous state S_{T-1} from the condition distribution

$$P(S_{t-1} = j|S_t = i, O_{1:t}, \theta) = \frac{P(S_{t-1} = j, S_t = i|O_{1:t}, \theta)}{P(S_t = i|O_{1:t}, \theta)} \tag{3.23a}$$

$$= \frac{P(S_t = i|S_{t-1} = j, \theta)P(S_{t-1} = j|O_{1:t}, \theta)}{P(S_t = i|O_{1:t}, \theta)} \tag{3.23b}$$

$$= \frac{\Gamma^t_{ji} u_{t-1|t-1}(j)}{u_{t|t}(i)} \tag{3.23c}$$

Corrected derivation. Let $\Gamma^{(t)}_{ji} = P(S_t = i \mid S_{t-1} = j, \theta)$ denote the transition from time $t-1$ to time t. We sample the last state S_T from the filtered distribution $u_{T|T}$, and then recursively, for $t = T, T-1, \ldots, 2$, sample S_{t-1} given the already sampled $S_t = i$ from the conditional distribution $P(S_{t-1} = j \mid S_t = i, O_{1:T}, \theta)$. By the Markov property, the future observations $O_{t:T}$ carry no further information about S_{t-1} once S_t is known, so

$$P(S_{t-1} = j \mid S_t = i, O_{1:T}, \theta) = P(S_{t-1} = j \mid S_t = i, O_{1:t-1}, \theta) \tag{3.24a}$$

$$= \frac{P(S_{t-1} = j, S_t = i \mid O_{1:t-1}, \theta)}{P(S_t = i \mid O_{1:t-1}, \theta)} \tag{3.24b}$$

$$= \frac{P(S_t = i \mid S_{t-1} = j, \theta)\, P(S_{t-1} = j \mid O_{1:t-1}, \theta)}{\sum_k P(S_t = i \mid S_{t-1} = k, \theta)\, P(S_{t-1} = k \mid O_{1:t-1}, \theta)} \tag{3.24c}$$

$$= \frac{\Gamma^{(t)}_{ji}\, u_{t-1|t-1}(j)}{\sum_k \Gamma^{(t)}_{ki}\, u_{t-1|t-1}(k)} = \frac{\Gamma^{(t)}_{ji}\, u_{t-1|t-1}(j)}{u_{t|t-1}(i)}, \tag{3.24d}$$

where $u_{t|t-1}(i) = P(S_t = i \mid O_{1:t-1}, \theta)$ is the one-step-ahead predictive state distribution. The method therefore only requires the filtered distributions from the forward pass and the transition matrices. Note that it still depends on the initial distribution π through the forward recursion, although this influence decays for large t.

3.2.4 Model Diagnostics

Goodness-of-fit

For asessing the goodnes of fit, we will use the pseudo-residuals defined in (chapter 7.3, Zucchini, MacDonald, and Langrock 2016). The pseudo-residuals are defined as

$$r_t = \Phi^{-1}\left(F_{t|t-1}(y_t \mid y_{1:t-1})\right), \tag{3.25}$$

where Φ^{-1} is the inverse of the standard normal cumulative distribution function (CDF) and $F_{t|t-1}(y_t \mid y_{1:t-1})$ is the one-step-ahead predictive CDF of the observation y_t given the previous observations $y_{1:t-1}$. The pseudo-residuals are expected to be independently and identically distributed (i.i.d.) standard normal if the model is correctly specified. Therefore, we can use the pseudo-residuals to assess the goodness-of-fit of the model by checking if they are normally distributed and uncorrelated.

Test Statistics for Comparison

For comparing the goodnes-of-fit of different models, we will use the Akaike information criterion (AIC), the Bayesian information criterion (BIC) and the Likelihood ratio test (LRT). Let $\hat{L}$ denote the maximised likelihood of a model, k the number of estimated parameters and T the number of observations.

The AIC (Wikipedia contributors 2026a) is defined as

$$\text{AIC} = 2k - 2\ln \hat{L}, \tag{3.26}$$

and the BIC (Wikipedia contributors 2026b) as

$$\text{BIC} = k\ln T - 2\ln \hat{L}. \tag{3.27}$$

Both criteria reward a high likelihood and penalise the number of parameters, and the model with the lowest value is preferred. Since $\ln T > 2$ for $T > 7$, the BIC penalises additional parameters harder than the AIC and therefore tends to favour simpler models.

The LRT (Wikipedia contributors 2026c) compares two nested models, where the null model with parameter space Θ_0 is a restricted version of the full model with parameter space Θ. The test statistic is

$$\lambda_{LR} = -2\left[\ell(\hat{\theta}_0) - \ell(\hat{\theta})\right], \tag{3.28}$$

where $\ell(\hat{\theta}_0)$ and $\ell(\hat{\theta})$ are the maximised log-likelihoods under the null and the full model, respectively. By Wilks' theorem, λ_{LR} is asymptotically χ^2-distributed with degrees of freedom equal to $\dim\Theta - \dim\Theta_0$, provided the null hypothesis lies in the interior of the parameter space. Note that this condition does not hold when comparing HMMs with different numbers of states, as the smaller model corresponds to parameters on the boundary of the larger model's parameter space (e.g. a transition probability equal to zero). The LRT is therefore only used for comparing nested models with the same number of states.

4 Implementation

This section provides an overview of what envoriment was used to implement the models, what libraries were used and why, how the classes were designed.

4.1 Coding envoriment

The implementation of the HMM models is in Python 3.11 and is split into two parts: a model library, which contains the HMMs themselves, and a set of scripts that clean the data, fit the models and evaluate them. The model library only depends on a small numerical stack:

- **JAX** (0.9.0) (Bradbury et al. 2018). All model computations, i.e. the emission densities, the transition matrices and the forward algorithm, are written in JAX. JAX provides automatic differentiation, so the gradient of the log-likelihood with respect to every model parameter is obtained without deriving it by hand. This is what allows the models to be fitted by direct numerical optimisation of the likelihood instead of the classical Baum-Welch (EM) algorithm, and it makes it cheap to add new emission or transition types, since no new update equations have to be derived. JAX also just-in-time compiles the code, and the recursion over time in the forward algorithm is written with `jax.lax.scan`, which keeps the fitting fast. Since the likelihood is numerically sensitive, all computations are done in 64-bit floating point.
- **Equinox** (0.13.7) (Kidger and Garcia 2021). Equinox lets the model classes be written as ordinary Python classes that are at the same time JAX pytrees. An HMM is therefore a single object whose parameters JAX can differentiate and optimise directly, and Equinox's filtering utilities are used to freeze selected parameters during a fit.
- **Optax** (0.2.8) (DeepMind 2020). Optax provides the optimisers that minimise the negative log-likelihood. The default is the quasi-Newton method L-BFGS, while first-order gradient methods such as Adam are available as an alternative.
- **NumPy** (2.2) (Harris, Millman, Walt, et al. 2020) for array handling outside of the JAX code.

The scripts around the library additionally use the following libraries:

- **pandas** (3.0) for loading, cleaning and aggregating the CO_2 time series into 30 minute intervals.
- **Matplotlib** (3.10) and **seaborn** (0.13) for all figures.
- **statsmodels** (0.14) for the autocorrelation functions and QQ-plots used to evaluate the pseudo-residuals of the fitted models.
- **holidays** (0.105) to mark Danish public holidays when constructing the time-of-day and weekday covariates.
- **Requests** (2.34) and **pyproj** (3.7) to download outdoor humidity data from the open data API of the Danish Meteorological Institute (DMI), where pyproj converts the coordinates of the building into the coordinate system used by DMI.

- **python-dotenv** (1.2) to read the location of the data from a local configuration file, so that the data itself is kept out of the code repository.

The package manager used is uv (Astral 2026). The exact versions of every library are recorded in a lock file, so the environment can be reproduced with a single `uv sync` command, and uv also installs the pinned Python version. This ensures that the results in this report can be reproduced with the same versions of the libraries that produced them.

4.2 Class Design

The class design of the model library is shown in fig. 4.1. An HMM is described by a `HMMParams` object, which is composed of a transition model and an emission model. Both are given as abstract base classes, `BaseTransition` and `BaseEmission`, with the concrete models from the theory chapter implemented as subclasses. Because the HMM is assembled from these two parts, a new model is obtained by combining an existing transition with an existing emission, e.g. a covariate-driven transition with an autoregressive emission, without writing any new code for the combination.

The remaining classes act on such a parameter object. The inference package contains the forward algorithm, which is used both by the negative log-likelihood that the solvers minimise, and by the diagnostics and the predictions on a fitted model. The solvers, the inference algorithm and the prediction are each given by an abstract base class, so an alternative optimiser or inference algorithm can be substituted without changing the model classes.

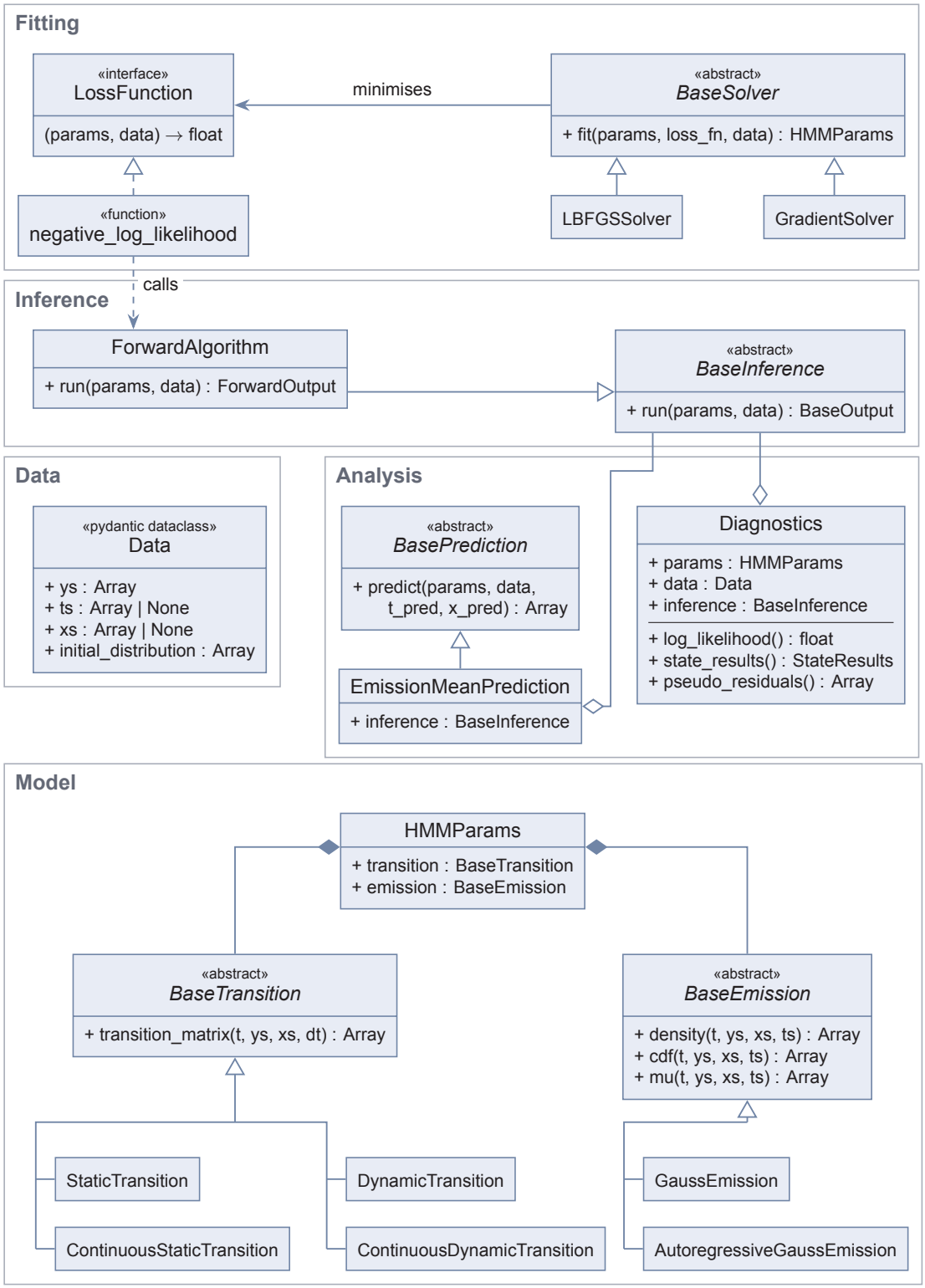


Figure 4.1: Class diagram of the model library, grouped into packages. Arrows with an open triangle denote inheritance (dashed: implementation of an interface), filled diamonds composition, open diamonds aggregation, and dashed arrows a call. Abstract classes are written in italics.

5 Model Results

6 Simulation Study

A

7 Discussion

8 Conclusion

9 Future Work

Bibliography

Batterman, Stuart (2017). “Review and Extension of CO_2-Based Methods to Determine Ventilation Rates with Application to School Classrooms”. In: *International Journal of Environmental Research and Public Health* 14.2, p. 145. doi: 10.3390/ijerph14020145.

Wang, Shengwei, John Burnett, and Hoishing Chong (1999). “Experimental Validation of CO_2-Based Occupancy Detection for Demand-Controlled Ventilation”. In: *Indoor and Built Environment* 8, pp. 377–391.

Iacus, Stefano M. (2008). *Simulation and Inference for Stochastic Differential Equations: With R Examples*. Springer Series in Statistics. New York: Springer. isbn: 978-0-387-75838-1. doi: 10.1007/978-0-387-75839-8.

Zucchini, Walter, Iain L. MacDonald, and Roland Langrock (2016). *Hidden Markov Models for Time Series: An Introduction Using R*. 2nd ed. Monographs on Statistics and Applied Probability 150. Boca Raton: Chapman and Hall/CRC. isbn: 978-1-4822-5383-2. doi: 10.1201/b20790.

Møller, Jan Kloppenborg et al. (2024). *Statistical Modelling of Occupant Behaviour*. London: Routledge. isbn: 978-1-032-33460-8.

Wikipedia contributors (2026a). *Akaike information criterion*. url: https://en.wikipedia.org/wiki/Akaike_information_criterion (visited on 11/04/2026).

– (2026b). *Bayesian information criterion*. url: https://en.wikipedia.org/wiki/Bayesian_information_criterion (visited on 11/04/2026).

– (2026c). *Likelihood-ratio test*. url: https://en.wikipedia.org/wiki/Likelihood-ratio_test (visited on 11/04/2026).

Bradbury, James et al. (2018). *JAX: composable transformations of Python+NumPy programs*. url: https://github.com/jax-ml/jax (visited on 11/04/2026).

Kidger, Patrick and Cristian Garcia (2021). “Equinox: neural networks in JAX via callable PyTrees and filtered transformations”. In: *Differentiable Programming workshop at Neural Information Processing Systems 2021*. arXiv: 2111.00254.

DeepMind (2020). *The DeepMind JAX Ecosystem*. url: https://github.com/google-deepmind/optax (visited on 11/04/2026).

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. (2020). “Array programming with NumPy”. In: *Nature* 585.7825, pp. 357–362. doi: 10.1038/s41586-020-2649-2.

Astral (2026). *uv: An extremely fast Python package and project manager*. url: https://docs.astral.sh/uv/ (visited on 11/04/2026).

A Proof of the AR(1) representation

Given our model of the CO_2 dynamics with noise in 3.6, we now want to show that the process can be described as a discrete-time AR(1) process.
Firstly, we recognize the diffusion process as a Ornstein-Uhlenbeck, and the solution of this process is given by Iacus (2008, Sec. 1.13.1) as

$$\tilde{C}(t) = \frac{P}{\alpha} + \left(\tilde{C}_0 - \frac{P}{\alpha}\right) e^{-\alpha t} + \sigma \int_0^t e^{-\alpha(t-u)} \, \mathrm{d}W_u \tag{A.1a}$$

or, in the second parametrization,

$$\tilde{C}(t) = \mu + (\tilde{C}_0 - \mu)e^{-\alpha t} + \sigma \int_0^t e^{-\alpha(t-u)} \, \mathrm{d}W_u, \qquad \mu = \frac{P}{\alpha}, \tag{A.1b}$$

where $\tilde{C}_0$ is the initial value of the process at time $t = 0$. μ should be interpretated as the equilibrium state of the process, which is the state that the process tends to oscillate around.

If we then discretize the process with a time step Δt, we can write the process as

$$\tilde{C}_{t+\Delta t} = \mu + (\tilde{C}_0 - \mu)e^{-\alpha(t+\Delta t)} + \sigma \int_0^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u, \tag{A.2a}$$

where we can split the integral into two parts as

$$\begin{aligned} \sigma \int_0^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u = \sigma \Bigg(& e^{-\alpha \Delta t} \int_0^t e^{-\alpha(t-u)} \, \mathrm{d}W_u \\ & + \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u \Bigg). \end{aligned} \tag{A.2b}$$

We can then write out the process as

$$\begin{aligned} \tilde{C}_{t+\Delta t} = \mu(1 - e^{-\alpha \Delta t}) + \mu e^{-\alpha \Delta t} + (\tilde{C}_0 - \mu)e^{-\alpha t} e^{-\alpha \Delta t} \\ + \sigma e^{-\alpha \Delta t} \int_0^t e^{-\alpha(t-u)} \, \mathrm{d}W_u + \sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u \end{aligned} \tag{A.2c}$$

We see then that

$$\begin{aligned} \tilde{C}_{t+\Delta t} &= \left(\mu + (\tilde{C}_0 - \mu)e^{-\alpha t} + \sigma \int_0^t e^{-\alpha(t-u)} \, \mathrm{d}W_u\right) e^{-\alpha \Delta t} \\ &\quad + \mu(1 - e^{-\alpha \Delta t}) + \sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u \\ &= \tilde{C}_t e^{-\alpha \Delta t} + \mu(1 - e^{-\alpha \Delta t}) + \sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u \end{aligned} \tag{A.2d}$$

We have now shown that diffusion process is dependent on the previous emission $\tilde{C}_t$.
But for it to be an AR(1) process, we nedd to show that the drift term has mean 0, constant

variance and is unoccorelated over time. [MISSING REFERENCE].
Let $\Delta t = c \in \mathbb{N}$ be a constant time step and given that $\alpha, t \in \mathbb{R}_{\geq 0}$, then from A.2d, we can see that the drift term is given by

$$\sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u = \tag{A.3}$$

We recognize $e^{-\alpha(t+\Delta t-u)}$ as our stochastic term, which is determinitics, meaning that that its ito intergratable as $\int_0^T e^{-2\alpha(t+\Delta t-u)} \, \mathrm{d}W_u < +\infty$.
Therefore, from Iacus (2008, Thm. 1.9.1), We now know that the mean of the drit term is

$$\mathbb{E}\left[\sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u\right] = 0 \tag{A.4}$$

And that the variance is

$$\mathrm{Var}\left[\sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u\right] = \sigma^2 \int_t^{t+\Delta t} e^{-2\alpha(t+\Delta t-u)} \, \mathrm{d}u \tag{A.5}$$

$$\begin{aligned}
\mathrm{Var}\left[\sigma \int_t^{t+\Delta t} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u\right] &= \sigma^2 \int_t^{t+\Delta t} e^{-2\alpha(t+\Delta t-u)} \, \mathrm{d}u && \text{(A.6a)} \\
&= \sigma^2 e^{-2\alpha(t+\Delta t)} \int_t^{t+\Delta t} e^{2\alpha u} \, \mathrm{d}u && \text{(A.6b)} \\
&= \sigma^2 e^{-2\alpha(t+\Delta t)} \left[\frac{1}{2\alpha} e^{2\alpha u}\right]_t^{t+\Delta t} && \text{(A.6c)} \\
&= \frac{\sigma^2}{2\alpha} e^{-2\alpha(t+\Delta t)} \left(e^{2\alpha(t+\Delta t)} - e^{2\alpha t}\right) && \text{(A.6d)} \\
&= \frac{\sigma^2}{2\alpha} \left(1 - e^{-2\alpha\Delta t}\right) && \text{(A.6e)}
\end{aligned}$$

We see that the variance is independent of t and that its constant over time.
We now only need to show that the covariance of the drift is 0.
We define that time steps as $t_1, t_2 \in \mathbb{R}_{\geq 0}$ such that $t_1 < t_2$ and that the time steps are disjoint, meaning that $[t_1, t_1 + \Delta t] \cap [t_2, t_2 + \Delta t] = \emptyset$ and that $t_1 + \Delta t = t_2$. We then define e_k as the drift term at time t_k such that

$$\begin{aligned}
\mathbb{E}\left[e_k\right] &= 0 \\
\mathrm{Var}(e_k) &= \sigma \int_{t_{k-1}}^{t_k} e^{-\alpha(t+\Delta t-u)} \, \mathrm{d}W_u = \frac{\sigma^2}{2\alpha}\left(1 - e^{-2\alpha\Delta t}\right)
\end{aligned} \tag{A.7}$$

We now then see that the covariance of the drift term is given by, given that $j \neq k$

$$\begin{aligned}
\mathrm{Var}(e_k + e_j) &= \mathrm{Var}(e_k) + \mathrm{Var}(e_j) + 2\mathrm{Cov}(e_k, e_j) \\
&\Leftrightarrow
\end{aligned} \tag{A.8}$$

To compute $\mathrm{Var}(e_k + e_j)$, we write both drift terms as integrals over the same interval $[0, T]$, where $T \geq \max(t_k, t_j) + \Delta t$, by introducing the deterministic functions

$$f_k(u) = \mathbb{1}_{[t_k,\, t_k+\Delta t)}(u)\, e^{-\alpha(t_k+\Delta t-u)}, \tag{A.9}$$

such that

$$e_k = \sigma \int_0^T f_k(u)\,\mathrm{d}W_u. \tag{A.10}$$

Since $j \neq k$, the intervals $[t_k, t_k + \Delta t)$ and $[t_j, t_j + \Delta t)$ are disjoint, and therefore

$$f_k(u) f_j(u) = 0 \quad \forall u \in [0, T]. \tag{A.11}$$

The sum $e_k + e_j = \sigma \int_0^T (f_k(u) + f_j(u))\,\mathrm{d}W_u$ is again an Itô integral with a deterministic integrand, so by Iacus (2008, Thm. 1.9.1)

$$\begin{aligned} \mathrm{Var}(e_k + e_j) &= \sigma^2 \int_0^T (f_k(u) + f_j(u))^2\,\mathrm{d}u \\ &= \sigma^2 \int_0^T f_k(u)^2\,\mathrm{d}u + \sigma^2 \int_0^T f_j(u)^2\,\mathrm{d}u + 2\sigma^2 \int_0^T f_k(u) f_j(u)\,\mathrm{d}u \\ &= \mathrm{Var}(e_k) + \mathrm{Var}(e_j) + 0. \end{aligned} \tag{A.12}$$

Comparing with $\mathrm{Var}(e_k + e_j) = \mathrm{Var}(e_k) + \mathrm{Var}(e_j) + 2\,\mathrm{Cov}(e_k, e_j)$ gives

$$\mathrm{Cov}(e_k, e_j) = 0, \qquad j \neq k. \tag{A.13}$$

Hence the drift terms have mean zero, constant variance and are uncorrelated, and the discretized process is an AR(1) process.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Asmussens Alle, Building 322
2800 Kgs. Lyngby
Tlf. 4525 1700

https://www.compute.dtu.dk